



BOOTCAMP FRONTEND & REACT · SEMANA 4

Datos reales

Clase 10 — fetch, promesas, async/await y manejo de errores. Cierra el bloque de JavaScript.

4 h 15 · L·M·V

Proyecto TechCart

STEER
RC
LA

PRIMERO CERRAMOS LA CLASE 9 · DESPUÉS, LOS DATOS DE VERDAD

Agenda

1. Repaso + el carrito se VE

estado → render

26'

2. localStorage

que sobreviva al refresco

36'

3. Taller: lista y quitar

delegación otra vez

26'

4. fetch y las promesas

el "vale por" un dato

40'

5. async/await + adaptador

38 productos reales en la tienda

38'

6. Errores y los 3 estados

cargando · error · vacío

36'

7. Promise.all y el plan B

en paralelo · respaldo local

34'

8. Cierre + 5 tareas

y el anticipo a TypeScript

19'

 **Receso de 15 min** entre el bloque 3 y el 4 · los bloques con franja naranja **cierran la Clase 9**

Repaso exprés · Clase 9

Dos por el chat

¿Por qué el listener va en el contenedor y no en cada botón?

¿Por qué `Number(boton.dataset.id)` y no directo `dataset.id`?

EL CARRITO YA FUNCIONA... PERO SOLO SE VE EN LA CONSOLA

Un dato, una función que pinta

carrito

el **dato**: la única fuente de verdad



click

el **evento** cambia el dato... y nada más



pintarCarrito()

el **render** mira el dato y lo dibuja

```
carrito = [...carrito, producto] // 1. cambio el dato
guardarCarrito(carrito) // 2. guardo
pintarCarrito() // 3. pinto
```



Caso real: cuando cada evento toca pedacitos de pantalla por su cuenta, la app termina **desincronizada**: el contador dice 3, la lista muestra 2 y el total no coincide con ninguno. Yo vi un carrito donde el precio del resumen y el del checkout no eran el mismo — dos lugares calculando por su lado.

SETITEM · GETITEM · REMOVEITEM —  UN MAP<STRING, STRING> QUE PERSISTE

localStorage solo guarda TEXTO

```
// ❌ lo que todos intentan primero
localStorage.setItem(CLAVE, carrito)
localStorage.getItem(CLAVE)
// "[object Object],[object Object]" ← basura
```

```
// ✅ siempre en pareja
localStorage.setItem(CLAVE,
JSON.stringify(carrito))
const guardado =
JSON.parse(localStorage.getItem(CLAVE))
```

stringify = de objeto a texto.

parse = de texto a objeto.

Y la clave en una **constante**: escrita a mano en cuatro lugares, basta una letra distinta para "perder" el carrito sin ningún error.

 **Caso real:** `JSON.parse` **revienta** con un JSON inválido (basura vieja, un guardado a medias). Y como corre al **arrancar**, la app **no abre** para ese usuario: pantalla en blanco imposible de reproducir en tu máquina.

TODO ACCESO AL ALMACENAMIENTO VA ENVUELTO

Cargar al arrancar, y **try/catch**

```
const cargarCarrito = () => {  
  try {  
    const crudo = localStorage.getItem(CLAVE)  
    return crudo ? JSON.parse(crudo) : []  
  } catch (error) {  
    console.warn("Carrito inválido, empiezo  
vacío")  
    return [] // la app ABRE, aunque pierda el  
carrito  
  }  
}  
  
let carrito = cargarCarrito()
```



Y al escribir también falla: `setItem` lanza excepción cuando el almacenamiento está **lleno** (~5 MB) y en **Safari en modo privado**. Sin `try/catch`, en un iPhone en navegación privada la tienda se cae al primer producto — y en tu máquina nunca lo vas a reproducir.

`sessionStorage`

la **misma API**, con una diferencia: se borra **al cerrar la pestaña**.



Nunca contraseñas ni datos sensibles: cualquier JS de la página lo lee.

EL MISMO MOVIMIENTO DEL CATÁLOGO: DATO → HTML → MAP + JOIN

La lista del carrito, y quitar

```
// ui.js - recibe el producto Y SU POSICIÓN
export const filaCarrito = ({ nombre, precio },
indice) => `
  <li>... <button data-accion="quitar" data-
indice="${indice}">
  Quitar</button></li>`

// main.js
listaCarrito.innerHTML =
carrito.map(filaCarrito).join("")

carrito = carrito.filter((producto, i) => i !==
indice)
```

map le entrega a su función el elemento **y su posición**. Es la primera vez que usamos ese segundo parámetro.

 **¿Por qué quito por posición y no por id?** Porque hoy el mismo producto puede entrar dos veces, y un filter por id borraría **las dos filas**: el usuario quita un producto y **desaparece otro**. Con cantidades (tarea difícil 2) vuelve a ser por id.

La segunda mitad

Cinco productos que escribí yo

Si mañana la tienda vende cuarenta, alguien tiene que venir a escribirlos. Si cambia un precio, lo mismo. Y si dos personas compran el último iPad al mismo tiempo, mi archivo no se enteró de nada.

Las tiendas de verdad piden sus productos a un **servidor**. Hoy TechCart va a traer **38 productos reales** desde una API por internet — y ahí aparece lo último que les falta de JavaScript: los datos **no llegan al instante**.

dummyjson.com/products/category/laptops → ábranlo en el navegador

LA PÁGINA NO PUEDE CONGELARSE ESPERANDO AL SERVIDOR

fetch devuelve un "vale por"

```
const productos = fetch("https://dummyjson.com/products/category/laptops")  
console.log(productos) // Promise { <pending> } ← NO son los productos
```

pending

el pedido salió, la respuesta viaja

fulfilled

llegó el dato

rejected

no se pudo hablar con el servidor

```
fetch(url)  
  .then(respuesta => respuesta.json()) // el SOBRE llegó; leer la carta es otra promesa  
  .then(datos => console.log(datos))
```

🔗 **Java:** es su `CompletableFuture`. Y `.json()` hace lo que hacía `JSON.parse`: todo lo que viaja por internet, viaja como **texto**.

TODO LO QUE DEPENDA DEL DATO TIENE QUE ESTAR ADENTRO

El error nº 1 con `fetch`

```
let productosDeLaAPI = []

fetch(url)
  .then(r => r.json())
  .then(datos => { productosDeLaAPI =
    datos.products })

console.log(productosDeLaAPI.length) // 0 ← ¿¿por qué??
```

El `console.log` **no esperó**: JavaScript mandó el pedido, siguió bajando por el archivo, imprimió el array vacío... y **después** llegó la respuesta.

🐛 **Caso real:** se ve con dos caras — `length` en 0 aunque el servidor respondió bien, o `Cannot read properties of undefined`. Y lo peor: a veces **funciona**, si la respuesta llega muy rápido. Sale bien una vez de cada diez en tu máquina con buen internet, y falla siempre en producción con un usuario en 3G.

Con dos pasos se aguanta. Con cinco, el código crece hacia la derecha: **callback hell**. Por eso existe lo que viene ahora.

LA MISMA PROMESA, LEÍDA DE ARRIBA ABAJO

async / await

CON .THEN

```
fetch(url)
  .then(r => r.json())
  .then(d => {
    console.log(d.products)
  })
```



CON ASYNC / AWAIT

```
const traerLaptops = async () => {
  const respuesta = await fetch(url)
  const datos = await respuesta.json()
  console.log(datos.products)
}
```

Cada resultado queda en una variable con nombre, sin anidar.

await → solo en async

fuera de una función async, el navegador se queja

async → devuelve promesa

aunque tú devuelvas un número

 no bloquea el hilo

parece que espera, pero la página sigue viva

Un traductor en la puerta

LA API DICE		TEHCART DICE
title	→	nombre
price	→	precio
thumbnail	→	imagen
category	→	categoría
brand ?? "Sin marca"	→	marca

```
// api.js - la capa de datos
const mapearProducto = (p) => ({
  id: p.id,
  nombre: p.title,
  precio: p.price,
  imagen: p.thumbnail,
  marca: p.brand ?? "Sin marca",
})

return datos.products.map(mapearProducto)
```

🚨 **Caso real:** sin adaptador, el día que la API cambie thumbnail por image hay que corregir **cuarenta archivos**. Con adaptador, **una línea**. Cuando algo puede cambiar, se lo encierra en un solo lugar.

LA TRAMPA MÁS CONTRAINTUITIVA DE FETCH

Un 404 no es un error para fetch

```
const respuesta = await
fetch(`${BASE}/productos`) // mal escrito
respuesta.status // 404
respuesta.ok // false
// ...y la promesa se CUMPLIÓ: el catch no se
disparó
datos.products // undefined → dos líneas después:
.map de undefined
```

```
if (!respuesta.ok) {
  throw new Error(`La tienda no respondió bien
(${respuesta.status}`)
}
```

fetch **solo** rechaza si no logró **hablar** con el servidor: sin internet, dominio inexistente, conexión cortada. Un 404 o un 500 **son respuestas**, y para él eso es éxito.

 **Caso real:** la app que se queda en "Cargando..." para siempre. El servidor devolvió 500, nadie miró el ok, el código se cayó dos líneas después y la función que apagaba el "Cargando" nunca se ejecutó. Para el usuario, la aplicación se colgó.

UN SOLO CATCH ATRAPA EL 404 Y LA RED CAÍDA

try / catch / finally

```
try {  
  productos = await obtenerProductos()  
  pintarCatalogo()  
} catch (error) {  
  contenedor.innerHTML = aviso(error.message)  
} finally {  
  // corre SIEMPRE: apagar un spinner,  
  // volver a habilitar un botón  
}
```

try

envuelve lo que puede fallar

catch

recibe el error —el mío del `throw` o el de la red— y **le habla al usuario** en vez de dejar la página muerta

finally

corre igual, salga bien o mal

 **Java:** idéntico al de ustedes.

PEDIR DATOS SON TRES CAMINOS, NO UNO

Cargando · error · vacío



Cargando

antes de pedir. Una página en blanco parece una página **rota**.



Error

el 404 o la red caída. Decir qué pasó, y que el resto siga vivo.



Vacío

respondió **200**... con un array **vacío**. No es un error: no hay nada.

```
contenedor.innerHTML = aviso("Cargando productos...")  
// ...  
if (productos.length === 0) return contenedor.innerHTML = aviso("No hay productos.")
```



Caso real: el estado vacío es el que se olvida siempre, y se nota el día que alguien usa el buscador. Escribe "zapatos" en una tienda de electrónica, no hay resultados, y la pantalla queda en blanco: la mitad de los usuarios cree que la página se rompió y se va.

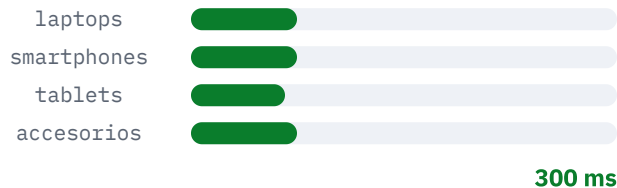
CUATRO CATEGORÍAS = CUATRO PEDIDOS INDEPENDIENTES

Promise.all: todos a la vez

❌ CUATRO AWAIT SEGUIDOS



✅ PROMISE.ALL



```
const CATEGORIAS = ["laptops", "smartphones",
  "tablets", "mobile-accessories"]

const listas = await Promise.all(
  CATEGORIAS.map(pedirCategoria)
)

return listas.flat() // 38 productos
```

Caso real: esa escalera se llama **waterfall**. El clásico: un bucle sobre 50 productos con un `await` adentro = 50 peticiones en fila, quince segundos de pantalla en blanco. **Cuando vean un `await` dentro de un bucle, sospechen.**

Si **una** falla, falla el conjunto y se va al `catch`. Para lo contrario: `Promise.allSettled`.

AHORA DEPENDEMOS DE UN SERVIDOR QUE NO CONTROLAMOS

Si la API se cae, la tienda **abre igual**

```
import { productos as respaldo } from "../datos.js"

export const obtenerProductos = async () => {
  try {
    const listas = await
    Promise.all(CATEGORIAS.map(pedirCategoria))
    return listas.flat()
  } catch (error) {
    console.warn("API caída, uso el respaldo
    local")
    return respaldo
  }
}
```

Los cinco productos que íbamos a tirar **no se tiran**: se convierten en el plan B.

El try/catch va **acá**, en `api.js`, y no en `main.js`: quien sabe qué hacer cuando los datos no llegan es la **capa de datos**, no la pantalla. `main.js` sigue igual, sin enterarse.

 **Caso real:** esto lo hacen las apps que usan todos los días. Spotify te muestra tu biblioteca sin conexión; el correo te deja leer lo ya bajado. Se llama **degradación elegante**: cuando algo falla, no se cae todo — se cae **menos**.

CON ESTO CERRAMOS TODO EL BLOQUE DE JAVASCRIPT

Resumen

estado → render

un dato, una función que pinta

localStorage

solo texto: stringify / parse + try/catch

fetch → Promise

un vale por un dato que aún no llegó

async / await

la misma promesa, de arriba abajo

el adaptador

traducir la API en UN solo lugar

response.ok

un 404 no es error para fetch

Y los **tres estados** —cargando, error, vacío— son parte del trabajo, no un extra. Más `Promise.all` para no hacer fila.

LA DIFÍCIL 2 LA NECESITO ARMADA PARA LAS CLASES DE REACT

5 tareas

FÁCIL

El contador en la pestaña: `document.title` con cuántos productos hay, desde `pintarCarrito`.

INTERMEDIO

Filtrar por categoría: la barra lateral, con delegación y `data-categoria`. Con 38 productos ya se nota.

INTERMEDIO

El aviso del respaldo: que la tienda diga "mostrando información guardada". Pista: devolver `{ productos, esRespaldo }`.

DIFÍCIL

El buscador: un `<input>` con el evento `input`, `filter + toLowerCase().includes()`, y el **estado vacío** si no hay resultados.

DIFÍCIL

Carrito con cantidades: cada producto una vez con cantidad; sube en vez de duplicar. Y quitar vuelve a ser por `id`.

Próxima clase

Y si escribo `p.title`?

No pasa **nada**. Ni un aviso, ni un subrayado, ni un error al correr: la tienda pinta **undefined** en las tarjetas y ustedes se pasan veinte minutos buscando dónde está el problema. Hoy nos pasó de verdad con la marca, y nos enteramos **mirando la pantalla**.

Esa es la incomodidad del **tipado dinámico** que arrastramos desde la Clase 7. La próxima clase llega **TypeScript**: le pone tipos a JavaScript y el editor avisa **antes de correr**. Para ustedes, que vienen de Java, va a ser como volver a casa.

Y hay una razón práctica: React se escribe en TypeScript — de la Clase 12 en adelante, nosotros también.